

Implementação de uma Linguagem de Domínio Específico para Comportamento de Carros Autônomos

Bruno da Silva Godoy

FHO – Araras

brunogodoy@alunos.fho.edu.br

Deibson dos Santos Lima

FHO – Araras

deibsons.official@alunos.fho.edu.br

Gabriel Rodrigues da Silva

FHO – Araras

gabriel.gabrielsilvaa@alunos.fho.edu.br

Abstract—Este trabalho apresenta o desenvolvimento de uma linguagem de domínio específico (DSL) para especificar o comportamento de veículos autônomos em um ambiente simulado. A DSL permite declarar sensores, escrever regras condicionais e acionar comandos de alto nível, como acelerar, frear, parar e virar. O interpretador foi implementado em Python, contemplando as fases de análise léxica, sintática, semântica e a execução das regras. A gramática formal da linguagem foi definida utilizando a notação BNF (Backus-Naur Form), classificada como gramática livre de contexto pertencente à Classe 2 da Hierarquia de Chomsky. Ao longo do artigo são descritos a gramática adotada, a classificação da linguagem e alguns resultados obtidos em cenários de teste.

Index Terms—linguagem de domínio específico, DSL, veículos autônomos, compiladores, gramática livre de contexto, interpretador, Python

I. INTRODUÇÃO

A. Relevância

O desenvolvimento de uma *Domain-Specific Language* (DSL) para definição de comportamentos de carros autônomos torna-se especialmente vantajoso quando comparado ao uso direto de linguagens de propósito geral, como Python ou C++. Embora Python ofereça alta produtividade e simplicidade sintática, sua natureza genérica exige que engenheiros de robótica lidem com detalhes de implementação que não pertencem diretamente ao domínio das regras de trânsito.

Uma DSL permite elevar o nível de abstração, oferecendo construções específicas para o contexto automotivo, possibilitando que o desenvolvedor descreva comportamentos utilizando comandos semânticos orientados ao domínio, como regras de parada, prioridade e manutenção de distância segura. Dessa forma, reduz-se a necessidade de manipular estruturas de controle complexas, bibliotecas externas ou lógica de baixo nível.

Além disso, a DSL permite validar as regras antes da execução, evitando configurações inválidas ou potencialmente perigosas. Em sistemas críticos, como veículos autônomos, esse controle sobre o que pode ser escrito na linguagem ajuda a deixar o sistema mais seguro e mais fácil de entender.

Nesse contexto, o Python será utilizado como linguagem hospedeira para a implementação do interpretador da DSL, aproveitando sua facilidade na construção de analisadores substituindo processos manuais de análise léxica e sintática,

enquanto a DSL fornece uma interface especializada e segura para definição de comportamentos.

B. Objetivos

A DSL proposta tem como objetivo permitir a especificação declarativa do comportamento de veículos autônomos por meio de regras de alto nível. As principais funcionalidades são:

- 1) **Leitura de sensores abstratos:** receber dados previamente processados provenientes dos sensores do veículo, como velocidade atual, distância de obstáculos, estado do semáforo e detecção de pedestres.
- 2) **Tomada de decisão baseada em regras:** permitir a criação de regras condicionais específicas do domínio de trânsito, possibilitando decisões automáticas baseadas no contexto do ambiente.
- 3) **Controle lógico de atuadores:** traduzir as decisões descritas na DSL em comandos de alto nível, como acelerar, frear, parar ou manter velocidade, que serão interpretados pelo sistema implementado em Python.

C. Escopo

Para garantir a viabilidade do projeto e manter o foco nos conceitos de compiladores e linguagens formais, o escopo da DSL será delimitado da seguinte forma:

- A linguagem **não realizará processamento direto de visão computacional ou dados brutos de sensores**.
- Apenas valores numéricos inteiros já interpretados por módulos externos serão utilizados como entrada (ex.: `distancia_frente = 5, velocidade = 0`).
- A DSL **não controlará hardware real**, funcionando apenas em ambiente simulado.
- Não serão implementados algoritmos de **aprendizado de máquina ou inteligência artificial** adaptativa.
- O projeto ficará na implementação do **interpretador em Python**, incluindo análise léxica, sintática, semântica e execução das regras definidas.

Essa delimitação permite concentrar o desenvolvimento nos conceitos centrais da disciplina, como definição de gramática, construção de *parser* e execução de uma linguagem específica de domínio.

II. PESQUISA BIBLIOGRÁFICA E ESTADO DA ARTE

A. Busca por Referências

A pesquisa bibliográfica teve como objetivo identificar trabalhos que dessem suporte teórico à criação de uma linguagem de domínio específico (DSL) aplicada ao comportamento de veículos autônomos, além de textos sobre gramáticas formais e construção de compiladores e interpretadores.

As referências foram levantadas principalmente em bases digitais de artigos científicos e em livros clássicos da área de compiladores e engenharia de software. A partir dessa busca, os materiais encontrados foram organizados em três grupos principais: (i) trabalhos que tratam de conceitos gerais de DSLs, suas vantagens e limitações; (ii) estudos que aplicam DSLs ou abordagens semelhantes em robótica e sistemas autônomos; e (iii) referências sobre gramáticas livres de contexto, análise léxica, sintática e semântica, que serviram como base para a definição da linguagem e do interpretador desenvolvidos.

Essas referências orientaram tanto o desenho da DSL (sintaxe, semântica e escopo) quanto as decisões de implementação das etapas de análise e execução da linguagem.

B. Análise Crítica

A literatura pesquisada confirma que o uso de linguagens de domínio específico é uma estratégia adequada para tratar problemas complexos com foco nos conceitos do domínio, sem expor o usuário à complexidade de linguagens de propósito geral.

Linguagens de domínio específico - fundamentos. Fowler (2010) apresenta uma visão abrangente sobre DSLs, descrevendo os diferentes tipos, benefícios e desafios de implementação. O autor destaca que, ao restringir a linguagem a um domínio bem definido, cria-se código mais legível e menos propenso a erros, pois o desenvolvedor utiliza diretamente a terminologia do domínio. Essa perspectiva sustenta a proposta deste trabalho, em que o comportamento de veículos autônomos é descrito por comandos como *acelerar*, *frear* e *parar*.

DSLs e linguagens hospedeiras. Hudak (1998) discute a construção modular de DSLs e sua relação com linguagens hospedeiras, mostrando como linguagens específicas podem ser desenvolvidas sobre infraestruturas já existentes. Esse modelo, conhecido como *embedded DSL*, é compatível com a abordagem adotada neste projeto, em que a DSL é interpretada por um sistema escrito em Python.

DSLs em robótica e sistemas automotivos. Côté et al. (2010) apresentam um framework voltado à prototipagem de sistemas automotivos, oferecendo abstrações de alto nível para especificar e testar comportamentos de veículos embarcados. O estudo ilustra como abstrações específicas facilitam o desenvolvimento em domínios complexos, reforçando a relevância de uma linguagem dedicada ao domínio de trânsito.

Segurança e formalização em veículos autônomos. Shalev-Shwartz, Shammah e Shashua (2017) propõem um

modelo formal de segurança para veículos autônomos, aumentando a necessidade de regras de comportamento explicitamente definidas e verificáveis. Esse argumento converge com a motivação deste trabalho: tornar as regras claras, estruturadas e verificáveis antes da execução.

Compiladores e gramáticas formais. Aho et al. (2007) fornecem a base teórica necessária para a implementação do interpretador, abordando gramáticas livres de contexto, análise léxica, sintática e semântica, além de tabelas de símbolos. Esses conceitos foram aplicados diretamente na definição da gramática em BNF, na classificação da DSL como livre de contexto (Classe 2 da hierarquia de Chomsky) e na arquitetura do interpretador em Python.

Comparando o estado da arte com a solução desenvolvida, destacam-se:

- A DSL delimita seu escopo a regras reativas simples em ambiente simulado, ao contrário de soluções que integram múltiplos módulos (percepção, planejamento, controle, comunicação).
- Foi projetada para ser **enxuta e didática**, servindo como ferramenta de apoio ao ensino de teoria da computação e compiladores.
- A implementação em Python privilegia a **clareza e modularidade** das fases de análise léxica, sintática e semântica, ao invés de otimizações de desempenho ou geração de código nativo.

No fim, a DSL que propomos reaproveita ideias que já são bem conhecidas na literatura, aplicadas aqui em um contexto acadêmico e em um recorte mais simples do problema de veículos autônomos.

C. Artigos Científicos Utilizados

O embasamento deste projeto estruturou-se a partir de referências fundamentais da literatura, destacando-se os seguintes trabalhos principais:

- 1) Fowler (2010) – Livro de referência sobre DSLs.
- 2) Hudak (1998) – Artigo sobre construção modular de DSLs.
- 3) Côté et al. (2010) – Framework para prototipagem de sistemas automotivos.
- 4) Shalev-Shwartz et al. (2017) – Modelo formal de segurança para veículos autônomos.
- 5) Aho et al. (2007) – Texto clássico sobre compiladores e gramáticas.

Essas referências cobrem a motivação conceitual, a base teórica de compiladores e a aplicação em veículos autônomos, trazendo uma fundamentação sólida para o desenvolvimento da linguagem proposta.

III. ESPECIFICAÇÃO DA LINGUAGEM

A. Gramática Formal da DSL

A linguagem proposta foi definida por meio de uma gramática formal utilizando a notação BNF (*Backus-Naur Form*). A BNF é amplamente utilizada na definição sintática

de linguagens formais e permite representar de maneira estruturada os elementos que compõem a DSL desenvolvida para especificação de comportamentos de carros autônomos.

O presente trabalho utiliza a notação BNF como base principal para a descrição formal da linguagem. A DSL suporta declarações de sensores, definição de regras, estruturas condicionais simples e comandos de ação do veículo. A estrutura geral do programa é composta por declarações de sensores e blocos de regras comportamentais, conforme descrito a seguir.

B. Estrutura Geral do Programa

O programa é composto por uma sequência de ações executadas de forma ordenada:

```
<programa> ::= <acao>
              | <acao> <programa>
```

As ações representam os comportamentos básicos do veículo e podem ser classificadas em quatro tipos principais:

```
<acao> ::= <movimento>
           | <controle>
           | <decisao>
           | <loop>
```

Os comandos de movimento permitem controlar diretamente a dinâmica do veículo:

```
<movimento> ::= "acelerar" "(" <valor> ")"
               | "frear" "(" <valor> ")"
               | "virar" "(" <direcao> "," <valor> ")"
```

Representam ações simples de controle:

```
<controle> ::= "parar"
               | "manter_velocidade"
```

A linguagem permite a definição de decisões condicionais simples ou compostas com suporte ao bloco "entao" e a opção de "senao":

```
<decisao> ::= "se" "(" <condicao> ")"
              "entao" "{" <lista_acoes> "}"
              | "se" "(" <condicao> ")"
              "entao" "{" <lista_acoes> "}"
              "senao" "{" <lista_acoes> "}"
```

Permite a execução contínua de ações enquanto uma condição for satisfeita:

```
<loop> ::= "enquanto" "(" <condicao> ")"
           "{" <lista_acoes> "}"
```

Define listas de ações dentro de blocos:

```
<lista_acoes> ::= <acao>
                  | <acao> <lista_acoes>
```

As condições são baseadas em sensores do ambiente e operadores relacionais:

```
<condicao> ::= <sensor> <operador> <valor>
```

```
<sensor> ::= "distancia_frente"
```

```
| "distancia_lateral"
| "velocidade"
| "obstaculo"
```

```
<operador> ::= "<"
              | ">"
              | "=="
              | "<="
              | ">="
```

Os valores utilizados nas expressões são definidos como números inteiros:

```
<valor> ::= <numero>
```

```
<numero> ::= <digito>
              | <digito> <numero>
```

```
<digito> ::= "0" | "1" | "2" | "3" | "4"
              | "5" | "6" | "7" | "8" | "9"
```

Define as direções possíveis para comandos de rotação:

```
<direcao> ::= "esquerda"
              | "direita"
```

C. Exemplo de Programa na DSL

A seguir apresenta-se um exemplo de uso da linguagem, demonstrando controle de velocidade, desvio de obstáculos e parada do veículo:

```
acelerar(30)
enquanto (distancia_frente > 5) {
    manter_velocidade
    se (distancia_lateral < 3) entao {
        virar(direita, 20)
    } senao {
        virar(esquerda, 10)
    }
}
frear(50)
parar
```

Descrição do Exemplo

O programa apresentado ilustra a execução de um comportamento autônomo modelado a partir da gramática definida em BNF. A estrutura segue o princípio de composição sequencial de ações, no qual o programa é interpretado como uma lista ordenada de comandos.

Inicialmente, o veículo executa uma ação de aceleração, estabelecendo uma velocidade base. Em seguida, é utilizada uma estrutura de repetição (*enquanto*), que introduz um comportamento contínuo dependente de uma condição baseada em sensores, neste caso, a *distancia_frente*.

Durante a execução do laço, o comando *manter_velocidade* é executado, representando um estado estável do sistema. Paralelamente, uma estrutura condicional (*se-entao-senao*) define decisões adaptativas com base na leitura do sensor *distancia_lateral*. Caso o veículo detecte proximidade

lateral, realiza uma correção mais acentuada na direção, caso contrário, aplica um ajuste mais suave.

Essa combinação de estruturas evidencia três conceitos fundamentais de gramáticas formais aplicadas à DSL:

- **Sequência:** a execução ordenada das ações no programa.
- **Escolha:** representada pelas estruturas condicionais.
- **Recursão:** presente na definição de blocos de ações repetitivos e aninhados.

Ao final da execução do laço, o veículo realiza uma desaceleração por meio do comando *frear(50)*, seguida da ação *parar*, encerrando o comportamento.

Dessa forma, a DSL demonstra capacidade de modelar comportamentos reativos e contínuos, nos quais decisões são tomadas dinamicamente a partir de entradas do ambiente, refletindo características essenciais de sistemas de condução autônoma.

D. Classificação na Hierarquia de Chomsky

A gramática que define a DSL foi especificada utilizando a notação BNF tradicional. Sua estrutura pode ser diretamente mapeada e classificada como uma gramática livre de contexto (CFG), pertencente à Classe 2 da Hierarquia de Chomsky.

Formalmente, uma gramática livre de contexto é definida por produções do tipo:

$$A \rightarrow \alpha$$

onde A é um único símbolo não terminal e α é uma sequência de símbolos terminais e/ou não terminais. Na BNF utilizada, construções estruturais complexas são resolvidas por meio de regras recursivas, o que preserva a propriedade de independência de contexto de forma estrita.

```
<lista_acoes> ::= <acao> | <acao>  
<lista_acoes>
```

A regra descrita acima, evidencia o uso de recursão à direita, típica de gramáticas livres de contexto para a definição de repetições ou listas de elementos.

Além disso, a sintaxe da DSL não apresenta dependências sensíveis ao contexto, como validações cruzadas complexas de tipos em tempo de compilação ou amarração de escopos dinâmicos estruturais externos à árvore sintática. Dessa forma, sua análise pode ser realizada com facilidade por parsers clássicos determinísticos, como os das famílias LL ou LR, amplamente documentados na literatura de compiladores.

Essa característica estrutural limpa permitiu a construção de um analisador sintático eficiente e a correta geração de uma árvore sintática abstrata (AST), utilizada diretamente pelo interpretador desenvolvido para simular os comportamentos do veículo.

E. Resultados

A DSL desenvolvida foi projetada com o objetivo de fornecer uma linguagem simples e especializada para controle de veículos ou sistemas que utilizem comandos direcionáveis. O foco principal do projeto não foi a construção de um simulador físico realista, mas sim a implementação de uma

estrutura funcional capaz de interpretar comandos de navegação, controle e tomada de decisão.

A arquitetura da aplicação foi dividida em diferentes módulos, responsáveis pela análise sintática, validação semântica, interpretação e simulação do ambiente. O parser foi desenvolvido utilizando a biblioteca PyParsing, responsável por transformar os comandos escritos pelo usuário em uma Árvore Sintática Abstrata (AST). Essa estrutura intermediária permitiu organizar os comandos da linguagem de forma hierárquica, facilitando sua interpretação posterior.

Entre os comandos implementados na DSL estão ações de movimentação, como aceleração, frenagem e alteração de direção, além de estruturas de controle condicionais e de repetição, como *se*, *senao* e *enquanto*. Essas estruturas possibilitam a criação de comportamentos dinâmicos para o veículo simulado, aproximando a linguagem de cenários reais de automação e navegação.

Após a etapa sintática, o sistema realiza uma análise semântica, responsável por validar parâmetros e garantir a integridade lógica dos comandos executados. Durante essa fase, o analisador verifica, por exemplo, se os sensores utilizados existem na linguagem, se os valores de aceleração são válidos e se as direções informadas pertencem ao conjunto permitido de comandos.

A execução da DSL ocorre por meio de um interpretador que percorre a AST e converte os comandos em ações simuladas dentro do ambiente. Durante a execução, o sistema mantém informações relacionadas ao estado do veículo, como velocidade, direção e distância de obstáculos.

Por se tratar de um projeto com foco na linguagem e não em simulações físicas complexas, parte das métricas ambientais foi simplificada. Alguns valores, como leitura de sensores e atualização de obstáculos, são gerados de maneira aleatória para permitir testes variados das estruturas da DSL sem a necessidade de implementar um motor físico completo. Essa abordagem permitiu validar o funcionamento da linguagem de maneira mais rápida e objetiva dentro do tempo disponível para o desenvolvimento do projeto.

```
2026-05-19 12:17:37.217 DEBUG    __main__:run_app:16 - Iniciando Parser (Análise Sintática)
2026-05-19 12:17:37.226 SUCCESS   __main__:run_app:19 - Sintaxe validada com sucesso!
2026-05-19 12:17:37.226 DEBUG    __main__:run_app:24 - Iniciando Analisador Semântico
2026-05-19 12:17:37.226 SUCCESS   __main__:run_app:28 - Semântica validada com sucesso!
2026-05-19 12:17:37.226 DEBUG    __main__:run_app:36 - Iniciando Interpretador e Simulação
2026-05-19 12:17:37.226 INFO      simulator:acelerar:16 - [CAR] Acelerando: +100 | Velocidade Atual: 100
2026-05-19 12:17:37.236 INFO      simulator:manter_velocidade:37 - [CAR] Mantendo velocidade atual: 100
2026-05-19 12:17:37.237 INFO      simulator:manter_velocidade:37 - [CAR] Mantendo velocidade atual: 100
2026-05-19 12:17:37.237 INFO      simulator:manter_velocidade:37 - [CAR] Mantendo velocidade atual: 100
```

Fig. 1. Saida

Mesmo utilizando simplificações, o sistema conseguiu demonstrar comportamentos coerentes durante a execução. Os logs gerados (Fig 1) mostram que o interpretador executa corretamente comandos sequenciais, condicionais e estruturas de repetição, além de reagir dinamicamente às alterações do ambiente simulado.

Também foi implementado um mecanismo de segurança para evitar loops infinitos durante a execução do comando *enquanto*. O interpretador possui um limite máximo de 100 iterações por loop, interrompendo automaticamente a execução

caso esse valor seja atingido. Esse limite pode ser ajustado conforme a necessidade da aplicação.

Os resultados obtidos demonstram que a DSL desenvolvida possui potencial para servir como base para sistemas de controle direcionável, podendo futuramente ser expandida para aplicações mais complexas, como robótica móvel, veículos autônomos ou ambientes de simulação mais avançados.

IV. CONCLUSÃO

Este trabalho apresentou o projeto, a formalização e a implementação de uma Linguagem de Domínio Específico (DSL) voltada para a especificação declarativa de comportamentos em veículos autônomos dentro de um ecossistema simulado. A modelagem sintática baseada estritamente na notação BNF provou-se altamente eficaz, permitindo traduzir ações complexas do domínio automotivo para uma estrutura livre de contexto estável, pertencente à Classe 2 da Hierarquia de Chomsky.

Os resultados obtidos demonstram que o objetivo principal do projeto foi plenamente alcançado. Através do desenvolvimento de um interpretador modular em Python, foi possível processar de forma bem-sucedida fluxos sequenciais de comandos, ramificações condicionais compostas e laços repetitivos reativos a dados de sensores abstratos organizados em uma Árvore Sintática Abstrata (AST). A lógica construída respondeu de maneira coerente e previsível aos parâmetros simulados de velocidade, direção e leitura de proximidade de obstáculos, provando o valor prático de se elevar o nível de abstração no controle robótico.

Durante o desenvolvimento do projeto, importantes desafios técnicos foram encontrados e superados. O primeiro deles consistiu em delimitar estritamente o escopo da linguagem, de forma a focar o projeto nos conceitos puros da teoria de compiladores, evitando o desvio para a complexidade matemática de motores físicos ou processamento de visão computacional. Outro desafio crucial envolveu o tratamento semântico das estruturas de repetição e o controle do ambiente, para contornar o risco de travamentos sistêmicos gerados por condições estáticas no comando *enquanto*, foi necessária a inclusão de um mecanismo de segurança restritivo limitando a execução a 100 iterações por laço. Adicionalmente, a estruturação manual da recursão à direita na BNF para garantir o aninhamento correto de blocos sem ambiguidade sintática exigiu testes rigorosos de validação.

Como sugestões de expansão e trabalhos futuros, aponta-se a evolução natural da gramática formal para o suporte a expressões lógicas compostas (utilizando operadores relacionais como *e*, *ou* e *nao*) e a manipulação de variáveis locais dinâmicas pelo usuário. No âmbito prático, recomenda-se a substituição do gerador de métricas simplificadas por uma integração direta do interpretador com motores de simulação física tridimensional consagrados na comunidade robótica, tais como o ROS (*Robot Operating System*) ou o simulador Webots. Dessa forma, as bases acadêmicas estabelecidas neste artigo poderão ser portadas para cenários industriais reais de teste de automação.

REFERENCES

- [1] FOWLER, Martin. *Domain-Specific Languages*. Boston: Addison-Wesley, 2010.
- [2] HUDAK, Paul. Modular Domain-Specific Languages and Tools. *IEEE Software*, v. 15, n. 4, p. 41–51, 1998. DOI: 10.1109/52.685237.
- [3] CÔTÉ, N. et al. A Framework for Automotive Software Prototyping. In: *IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*, 2010, p. 1–8. IEEE.
- [4] SHALEV-SHWARTZ, Shai; SHAMMAH, Shaked; SHASHUA, Amnon. On a Formal Model of Safe and Scalable Self-driving Cars. *arXiv preprint arXiv:1708.06374*, 2017.
- [5] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. Boston: Pearson, 2007.
- [6] ASHENDEN, Peter J.; PETERSON, Gregory D.; TEEGARDEN, Darrell A. *The System Designer's Guide to VHDL-AMS*. San Francisco: Morgan Kaufmann, 2003. Capítulo 1: Fundamental Concepts, p. 1–37. Disponível em: <https://www.sciencedirect.com/science/article/pii/B9781558607491500013>. Acesso em: 28 abr. 2026.
- [7] B. S. Godoy, D. S. Lima e G. R. Silva. “Projeto DSL: Implementação de uma Linguagem de Domínio Específico para Comportamento de Carros Autônomos,” GitHub. em: https://github.com/DeibsonS/Projeto_DSL.git.